Department of

Artificial Intelligence & Machine Learning



B.Tech III Semester

Academic Year 2026-2027

Data Base Management Systems (25B11CS001)

Capstone Stone Project Documentation

ADITYA UNIVERSITY

Surampalem, Kakinada District

Andhra Pradesh-533437

ONLINE SHOPPING SYSTEM

A DBMS Capstone Project Report submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted By

A.SHANKAR (25B11AI042), CH.MANOHAR (25B11AI185),G.RAKESH VYSHNAV (25B11AI353), T.KALEBU (25B11AIB74)

Under the esteemed guidance of

Mrs. G L Narasamba vanguri M.Tech.,(Ph.D)

ASSISTANT PROFEESOR

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

ADITYA UNIVERSITY

(Formerly Aditya Engineering College (A))

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

2026 – 2027

SCHOOL OF COMPUTING

Department of AI&ML

CERTIFICATE

This is to certify that the Cornerstone project work entitled *"ONLINE SHOPPING SYSTEM"* is submitted by

A.SHANKAR	**(25B11AI042)**
CH.MANOHAR	**(25B11AI185)**
G.RAKESH VYSHNAV	**(25B11AI353)**
T.KALEBU	**(25B11AIB74)**

in partial fulfillment of the requirements for the award of the B.Tech. degree in AI&ML for the academic year 2026-2027.

Project Guide

Mrs. G L Narasamba vanguri , M.Tech.,(Ph.D),
Assistant Professor,
Department of AI&ML

Head of the Department

Dr.Naga bhargavi, Ph.D,
Associate Professor,
Department of AI&ML

DECLARATION

We hereby declare that the Cornerstone project entitled "**ONLINE SHOPPING SYSTEM**" is original and is submitted to **ADITYA UNIVERSITY**, Surampalem, in partial fulfillment of the requirements for the award of the **B.Tech. Degree** in **Artificial intelligence and machine learning**. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy

Place: Surampalem

Date:

Submitted By:

A.SHANKAR	**(25B11AI042)**
CH.MANOHAR	**(25B11AI185)**
G.RAKESH VYSHNAV	**(25B11AI353)**
T.KALEBU	**(25B11AIB74)**

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Mrs. G L Narasamba vanguri ,M.Tech.,(Ph.D),, Assistant Professor,** who has guided us throughout the project. His valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **DR. R.SUNEETHA ,ASSISTANT PROFESSOR , DR B MANIKYALA RAO, ASSISTANT PROFESSOR** , Department of Data Scinece, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **DR. K NAGA BHARGAVI ASSOCIATE PROFESSOR,** for his inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Computing** for their encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas, Vice-Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

ADITYA UNIVERSITY

Surampalem

Department of Computer Science and Engineering

DBMS CORNERSTONE PROJECT

PROJECT DOCUMENTATION

ONLINE SHOPPING SYSTEM

Database Design, Normalization and SQL Implementation

Guidance of

Mrs. G L Narasamba vanguri M.Tech.,(Ph.D),
Assistant professor

presented by

A.SHANKAR
[25B11AI042]

CH.MANOHAR
[25B11AI185]

G.RAKESH VYSHNAV
[25B11AI353]

T.KALEBU
[25B11AIB74]

Course: Database Management System(DBMS) Course Code:DBMS

Faculty coordinator:

Academic Year : 2026-27 Date of Submission:6/10/2026

Repository Link : https://github.com/hemanthmindrana2008-spec/Online Shopping-order-management-system/tree/main

TABLE OF CONTENTS

Chapter	Title	Page No.
	Certificate	i
	Declaration	ii
	Acknowledgement	iii
	Project Documentation Report	iv
1	**ABSTRACT**	v
	List of Figures	vii
	List of Tables	vii
2	**INTRODUCTION**	1
	2.1 Background	1
	2.2 Purpose of the Database	1
3	**PROBLEM STATEMENT**	1
4	**OBJECTIVES**	1
5	**TOOLS & TECHNOLOGIES USED**	2
6	**DATABASE DESIGN**	2
	6.1 Identified Entities	2
	6.2 Entity Attributes	2
	6.3 ER Diagram	4
	6.4 Relationships & Cardinality	4
	6.5 Relational Database Schema	4
	6.6 Normalization	6
7	**DATABASE IMPLEMENTATION**	8
	7.1 DDL — Database and Table Creation	8
	7.2 Sample Data — DML.	10
	7.3 Constraints & Data Integrity	10
	7.4 SQL Operations & Queries	10
	7.5 Sample Outputs / Screenshots	12
8	**MODULES & FUNCTIONALITIES**	14
9	**GITHUB REPOSITORY & TEAM RESPONSIBILITIES**	14
10	**ENHANCEMENTS & FUTURE IMPROVEMENTS**	15
11	**CONCLUSION**	15
12	**REFERENCES**	15
	APPENDIX	16
	A. Complete SQL Script	16
	B. Additional Screenshots / Outputs	16

LIST OF FIGURES

Figure No.	Title of the Figure	Page No.
6.3	ER Diagram	15
Figure 1	Customer records / basic SELECT output	26
Figure 2	Active savings accounts / filtering output	28
Figure 3	Aggregate or GROUP BY output	30
Figure 4	Join or subquery output	34

LIST OF TABLES

Table No.	Title of the Table	Page No.
5	TOOLS & TECHNOLOGIES USED	2
6.1	Identified Entities	13
6.2	Entity Attributes	13
6.4	Relationships & Cardinality	14
6.5	Relational Database Schema	16
7.2	Sample Data — DML.	22-23
7.3	Constraints & Data Integrity	18
7.4	SQL Operations & Queries	25
8	MODULES & FUNCTIONALITIES	20
9	GITHUB REPOSITORY & TEAM RESPONSIBILITIES	16

ABSTRACT

The Online Shopping System is a centralized relational database designed to manage customers, product categories, products, orders, order items, payments, and shipments. The system provides a structured approach to storing and retrieving shopping information while maintaining relationships between business entities. Primary keys, foreign keys, uniqueness rules, NOT NULL conditions and CHECK constraints help maintain data integrity. The database design is organized toward Third Normal Form (3NF), reducing redundancy and update anomalies. SQL operations support product lookup, customer management, order processing, sales reporting and shipment tracking.

ACRONYMS

Acronym	Meaning
DBMS	Database Management System
ER	Entity Relationship
SQL	Structured Query Language
DDL	Data Definition Language
DML	Data Manipulation Language
DQL	Data Query Language
PK	Primary Key
FK	Foreign Key
1NF	First Normal Form
2NF	Second Normal Form
3NF	Third Normal Form

1. INTRODUCTION

1.1 Brief Information about the Project

An Online Shopping System is a database application that manages the core information and transactions of an online shopping platform. It maintains customer records, product categories, product details, orders, order items, payments and shipments in a centralized relational database.

1.2 Motivation and Contribution of Project

- Centralizing online shopping records in one relational database.
- Reducing duplicate and inconsistent customer, product and order information.
- Maintaining referential integrity using primary and foreign keys.
- Supporting efficient order searches, sales summaries and operational reports.
- Providing a foundation for order, payment, inventory and shipment management.

1.3 Objectives of the Project

- Develop a centralized online shopping order database.
- Manage customer and product information.
- Create and track customer orders and order items.
- Record payment information and payment status.
- Maintain shipment and delivery information.
-
- Reduce duplicate data and update anomalies.
- Support SQL queries and business reports.

1.4 Scope of the Project

The scope covers customer management, product and category management, order processing, order-item management, payment records, shipment tracking and SQL-based reporting. The system can later be extended with inventory reservation, discounts, returns, refunds, reviews, notifications, analytics and role-based access.

2. SYSTEM ANALYSIS

2.1 Existing System

- Manual or spreadsheet-based order maintenance.
- Customer and product information stored in disconnected files.
- Repeated order and product information.
- Time-consuming searches and report preparation.
- Difficulty tracking order and shipment status.
- Higher possibility of data-entry errors.
-

2.2 Proposed System

The proposed system uses a centralized relational database. It organizes online shopping information into related tables and uses primary keys, foreign keys, uniqueness rules, NOT NULL conditions and CHECK constraints. The system supports customer, category, product, order, payment and shipment management.

- Centralized database
- Customer management
- Product management
- Order management
- Payment processing
- Shipment tracking
- SQL-based reporting
- Referential integrity

2.3 Functional Requirements

Module	Functional Requirement
Customer Management	Add, update, search and display customer details.
Category Management	Store and maintain product category information.
Product Management	Maintain product name, price, stock and category.
Order Management	Create orders and maintain order date, status and total.
Order Item Management	Store products, quantities and unit prices for each order.
Payment Management	Record payment method, amount and payment status.
Shipment Management	Store shipping address and delivery status.

Search and Reporting	Retrieve customer, order, product and sales information using SQL.

2.4 Non-Functional Requirements

- Performance – queries should return results efficiently for normal online shopping workloads.
- Integrity – invalid references and duplicate key values should be prevented.
- Security – database privileges should be controlled for authorized users.
-
- Usability – forms and reports should be understandable to operational users.
- Reliability – transactions should preserve a consistent database state.
- Scalability – the design should support growth in customers, products and orders.

2.5 Requirements Specification

The requirements specification defines the functional and non-functional needs of the Online Shopping System. It provides a clear basis for database design, SQL implementation and validation of the proposed system.

3. DATABASE DESIGN

3.1 Major Entities

Entity	Description
Customer	Stores customer identity and contact information.
Category	Stores product category information.
Product	Stores products, prices, stock and category reference.
Orders	Represents a customer's order transaction.
Order_Item	Represents products and quantities within an order.
Payment	Stores payment details associated with an order.
Shipment	Stores shipping and delivery details associated with an order.

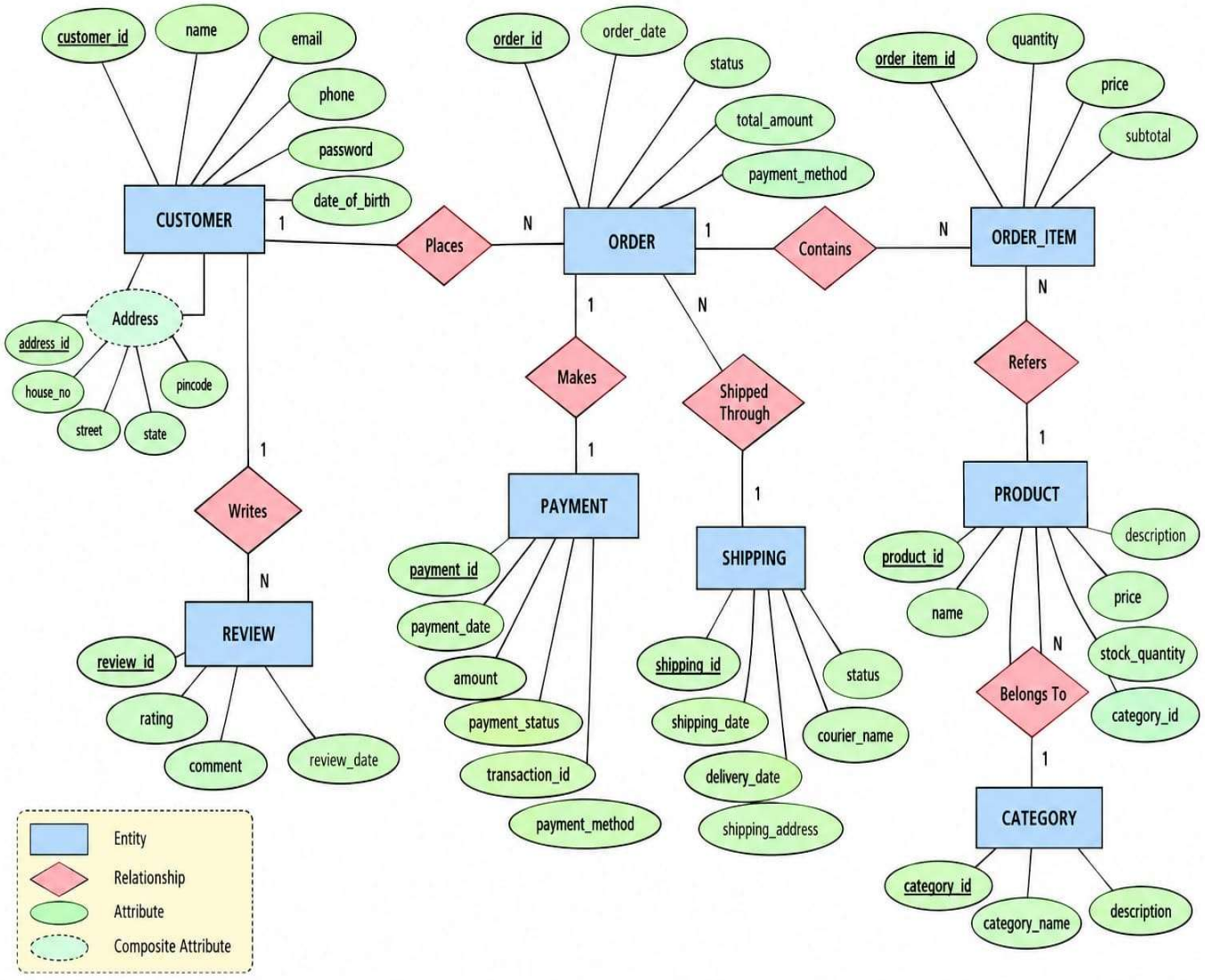
3.2 ER Model

The ER model represents Customer, Category, Product, Orders, Order_Item, Payment and Shipment entities. Customer is related to Orders; Orders are related to Order_Item, Payment and Shipment; Order_Item references Product; and Product references Category.

3.3 Relationships

Relationship	Description
Customer → Orders	One customer can have many orders.
Category → Product	One category can contain many products.
Product → Order_Item	One product can appear in many order items.
Orders → Order_Item	One order can contain many order items.
Orders → Payment	An order can have an associated payment record.
Orders → Shipment	An order can have an associated shipment record.

4. ER DIAGRAM



Recommended diagram flow: CUSTOMER 1—M ORDERS; ORDERS 1—M ORDER_ITEM; PRODUCT 1—M ORDER_ITEM; CATEGORY 1—M PRODUCT; ORDERS 1—1/0 PAYMENT; ORDERS 1—1/0 SHIPMENT.

5. RELATIONAL SCHEMA

Relation	Relational Schema
CUSTOMER	Customer_ID (PK), Customer_Name, Email, Phone_No, Address
CATEGORY	Category_ID (PK), Category_Name
PRODUCT	Product_ID (PK), Product_Name, Price, Stock_Qty, Category_ID (FK)
ORDERS	Order_ID (PK), Customer_ID (FK), Order_Date, Order_Status, Total_Amount
ORDER_ITEM	Order_Item_ID (PK), Order_ID (FK), Product_ID (FK), Quantity, Unit_Price
PAYMENT	Payment_ID (PK), Order_ID (FK), Payment_Date, Amount, Payment_Method, Payment_Status
SHIPMENT	Shipment_ID (PK), Order_ID (FK), Shipping_Address, Shipment_Date, Delivery_Status

Team Responsibilities table

Student Name	Roll number	Responsibility	Contribution Status
M.HEMANTH BABU	25B11AI739	Presentation outline ,Design, github respo, Project documentation	Completed
G SHARA	26B21AI038	Entities & attributes ,ER Diagram, data insertion Demonstrate & present	Completed
P S D S KRISHNA	26B21AI097	Data Schema ,Pk&Fk ,Crud & SQL queries, Data integrity	Completed
CH TEJA SAI DURGA	25B11AI203	Database & constraints, DDL ,DQL ,DML, Conclusion	Completed

Repository Link : https://github.com/hemanthmindrana2008-spec/Online Shopping-order-management-system/tree/main

6. TABLE DESIGN

6.1 Customer Table

Attribute	Data Type	Constraint	Description
Customer_ID	NUMBER	PK, NOT NULL	Unique customer identifier
Customer_Name	VARCHAR2	NOT NULL	Customer name
Email	VARCHAR2	UNIQUE	Customer email
Phone_No	VARCHAR2	—	Contact number
Address	VARCHAR2	—	Customer address

6.2 Category Table

Attribute	Data Type	Constraint	Description
Category_ID	NUMBER	PK, NOT NULL	Unique category identifier
Category_Name	VARCHAR2	UNIQUE, NOT NULL	Category name

6.3 Product Table

Attribute	Data Type	Constraint	Description
Product_ID	NUMBER	PK	Unique product identifier
Product_Name	VARCHAR2	NOT NULL	Product name
Price	DECIMAL	CHECK >= 0	Selling price
Stock_Qty	NUMBER	CHECK >= 0	Available stock quantity
Category_ID	NUMBER	FK	References Category

6.4 Orders Table

Attribute	Data Type	Constraint	Description
Order_ID	NUMBER	PK	Unique order identifier
Customer_ID	NUMBER	FK, NOT NULL	References Customer
Order_Date	DATE	DEFAULT	Date order was placed
Order_Status	VARCHAR2	—	Placed, shipped, delivered, cancelled, etc.
Total_Amount	DECIMAL	CHECK >= 0	Total order value

6.5 Order Item Table

Attribute	Data Type	Constraint	Description
Order_Item_ID	NUMBER	PK	Unique line-item identifier
Order_ID	NUMBER	FK	References Orders
Product_ID	NUMBER	FK	References Product
Quantity	NUMBER	CHECK > 0	Quantity ordered
Unit_Price	DECIMAL	CHECK >= 0	Price at time of order

6.6 Payment Table

Attribute	Data Type	Constraint	Description
Payment_ID	NUMBER	PK	Unique payment identifier
Order_ID	NUMBER	FK	References Orders
Payment_Date	DATE	—	Payment date
Amount	DECIMAL	CHECK >= 0	Payment amount
Payment_Method	VARCHAR2	—	Card, UPI, COD, etc.
Payment_Status	VARCHAR2	—	Pending, Paid, Failed, Refunded

6.7 Shipment Table

Attribute	Data Type	Constraint	Description
Shipment_ID	NUMBER	PK	Unique shipment identifier
Order_ID	NUMBER	FK	References Orders
Shipping_Address	VARCHAR2	—	Delivery address
Shipment_Date	DATE	—	Shipment date
Delivery_Status	VARCHAR2	—	Processing, Shipped, Delivered, Returned

7. KEYS AND CONSTRAINTS

Constraint Type	Applied To / Purpose
Primary Key	Customer_ID, Category_ID, Product_ID, Order_ID, Order_Item_ID, Payment_ID, Shipment_ID
Foreign Key	Links Product→Category, Orders→Customer, Order_Item→Orders/Product, Payment/Shipment→Orders
NOT NULL	Required names and identifiers
UNIQUE	Customer email and category name where applicable
CHECK	Price and quantity ranges, valid amounts and business rules
DEFAULT	Order date or default status where appropriate

8. NORMALIZATION

Normalization organizes data into related relations to reduce redundancy and update anomalies. The online shopping order example can be described from UNF to 3NF.

8.1 Unnormalized Form (UNF)

A single order record might contain customer details, repeated product names, quantities, prices, payment information and shipping information. Repeating product groups and duplicated customer/category information create redundancy.

8.2 First Normal Form (1NF)

In 1NF, repeating groups are removed and each attribute contains atomic values. Each product within an order is represented by a separate order-item row.

8.3 Second Normal Form (2NF)

In 2NF, partial dependencies are removed. Customer attributes depend on Customer_ID, product attributes depend on Product_ID, and order-specific attributes depend on Order_ID. This motivates separate Customer, Product, Orders and Order_Item relations.

8.4 Third Normal Form (3NF)

In 3NF, transitive dependencies are removed. Category information is stored in Category, customer information in Customer, product information in Product, payment information in Payment and shipping information in Shipment. The resulting structure supports the 3NF objective.

Relation	Reason for Separation
Customer	Stores customer-specific attributes once.
Category	Stores category-specific attributes once.
Product	Stores product-specific attributes and Category_ID.
Orders	Stores transaction-specific attributes and Customer_ID.
Order_Item	Stores the order-product relationship, quantity and unit price.
Payment	Stores payment-specific information linked to an order.
Shipment	Stores delivery-specific information linked to an order.

9. SQL IMPLEMENTATION

9.1 DDL – Data Definition Language

```
CREATE TABLE Customer (
  Customer_ID NUMBER PRIMARY KEY,
  Customer_Name VARCHAR2(100) NOT NULL,
  Email VARCHAR2(120) UNIQUE,
  Phone_No VARCHAR2(20),
  Address VARCHAR2(250)
);

CREATE TABLE Category (
  Category_ID NUMBER PRIMARY KEY,
  Category_Name VARCHAR2(100) UNIQUE NOT NULL
);

CREATE TABLE Address (

   address_id NUMBER PRIMARY KEY,

   customer_id NUMBER NOT NULL,

   address_line VARCHAR2(200) NOT NULL,

   city VARCHAR2(60) NOT NULL,

   state VARCHAR2(60),

   postal_code VARCHAR2(15) NOT NULL,

   country VARCHAR2(60) NOT NULL,

   CONSTRAINT fk_address_customer

      FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)

);



CREATE TABLE Product (
  Product_ID NUMBER PRIMARY KEY,
  Product_Name VARCHAR2(150) NOT NULL,
  Price NUMBER(10,2) CHECK (Price >= 0),
  Stock_Qty NUMBER CHECK (Stock_Qty >= 0),
  Category_ID NUMBER,
  CONSTRAINT fk_product_category
    FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID)
);
```

```
CREATE TABLE Orders (
  Order_ID NUMBER PRIMARY KEY,
  Customer_ID NUMBER NOT NULL,
  Order_Date DATE DEFAULT SYSDATE,
  Order_Status VARCHAR2(30),
  Total_Amount NUMBER(12,2) CHECK (Total_Amount >= 0),
  CONSTRAINT fk_order_customer
    FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
);



CREATE TABLE Order_Item (
  Order_Item_ID NUMBER PRIMARY KEY,
  Order_ID NUMBER NOT NULL,
  Product_ID NUMBER NOT NULL,
  Quantity NUMBER CHECK (Quantity > 0),
  Unit_Price NUMBER(10,2) CHECK (Unit_Price >= 0),
  CONSTRAINT fk_item_order FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID),
  CONSTRAINT fk_item_product FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)
);

      CREATE TABLE Payment (
        Payment_ID NUMBER PRIMARY KEY,
        Order_ID NUMBER NOT NULL,
        Payment_Date DATE,
        Amount NUMBER(12,2) CHECK (Amount >= 0),
        Payment_Method VARCHAR2(30),
        Payment_Status VARCHAR2(30),
        CONSTRAINT fk_payment_order FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)
      );

      CREATE TABLE Shipment (
        Shipment_ID NUMBER PRIMARY KEY,
        Order_ID NUMBER NOT NULL,
        Shipping_Address VARCHAR2(250),
        Shipment_Date DATE,
        Delivery_Status VARCHAR2(30),
        CONSTRAINT fk_shipment_order FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)
      );
```

9.2 DML – Sample Data

1.Customer

```
INSERT INTO Customer (Customer_ID,Customer_Name,Email,Phone_No,Address)
VALUES (1,'Rahul Kumar','rahul@gmail.com','9876543210','Kakinada, Andhra Pradesh');

INSERT INTO Customer (Customer_ID,Customer_Name,Email,Phone_No,Address)
VALUES (2,'Priya Sharma','priya@gmail.com','9876543211','Visakhapatnam, Andhra Pradesh');

INSERT INTO Customer (Customer_ID,Customer_Name,Email,Phone_No,Address)
VALUES (3,'Arjun Reddy','arjun@gmail.com','9876543212','Vijayawada, Andhra Pradesh');

INSERT INTO Customer (Customer_ID,Customer_Name,Email,Phone_No,Address)
VALUES (4,'Sneha Rao','sneha@gmail.com','9876543213','Rajahmundry, Andhra Pradesh');

INSERT INTO Customer (Customer_ID,Customer_Name,Email,Phone_No,Address)
VALUES (5,'Kiran Kumar','kiran@gmail.com','9876543214','Hyderabad, Telangana');
```

2.Category

```
INSERT INTO Category (Category_ID,Category_Name)
VALUES (1,'Electronics');

INSERT INTO Category (Category_ID,Category_Name)
VALUES (2,'Clothing');

INSERT INTO Category (Category_ID,Category_Name)
VALUES (3,'Books');

INSERT INTO Category (Category_ID,Category_Name)
VALUES (4,'Home Appliances');

INSERT INTO Category (Category_ID,Category_Name)
VALUES (5,'Sports');
```

3.Address

```
INSERT INTO Address (address_id,customer_id,address_line,city,state,postal_code,country)
VALUES (101,1,'12 Main Road','Kakinada','Andhra Pradesh','533001','India');

INSERT INTO Address (address_id,customer_id,address_line,city,state,postal_code,country)
VALUES (102,2,'25 Beach Road','Visakhapatnam','Andhra Pradesh','530001','India');

INSERT INTO Address (address_id,customer_id,address_line,city,state,postal_code,country)
VALUES (103,3,'45 MG Road','Vijayawada','Andhra Pradesh','520001','India');

INSERT INTO Address (address_id,customer_id,address_line,city,state,postal_code,country)
VALUES (104,4,'10 Station Road','Rajahmundry','Andhra Pradesh','533101','India');



INSERT INTO Address (address_id,customer_id,address_line,city,state,postal_code,country)
```

VALUES (105,5,'18 Banjara Road','Hyderabad','Telangana','500034','India');

4.Product

INSERT INTO Product (Product_ID,Product_Name,Price,Stock_Qty,Category_ID)
VALUES (201,'Wireless Headphones',2499,50,1);

INSERT INTO Product (Product_ID,Product_Name,Price,Stock_Qty,Category_ID)

VALUES (202,'Smart Watch',3999,30,1);

INSERT INTO Product (Product_ID,Product_Name,Price,Stock_Qty,Category_ID)
VALUES (203,'Cotton T-Shirt',799,100,2);

INSERT INTO Product (Product_ID,Product_Name,Price,Stock_Qty,Category_ID)
VALUES (204,'Java Programming Book',650,25,3);

INSERT INTO Product (Product_ID,Product_Name,Price,Stock_Qty,Category_ID)
VALUES (205,'Electric Kettle',1499,8,4);

INSERT INTO Product (Product_ID,Product_Name,Price,Stock_Qty,Category_ID)
VALUES (206,'Cricket Bat',2499,15,5);

5.Orders

INSERT INTO Orders (Order_ID,Customer_ID,Order_Date,Order_Status,Total_Amount)
VALUES (301,1,TO_DATE('2026-09-01','YYYY-MM-DD'),'PLACED',2499);

INSERT INTO Orders (Order_ID,Customer_ID,Order_Date,Order_Status,Total_Amount)
VALUES (302,2,TO_DATE('2026-09-02','YYYY-MM-DD'),'PAID',3999);

INSERT INTO Orders (Order_ID,Customer_ID,Order_Date,Order_Status,Total_Amount)
VALUES (303,3,TO_DATE('2026-09-03','YYYY-MM-DD'),'SHIPPED',1449);

INSERT INTO Orders (Order_ID,Customer_ID,Order_Date,Order_Status,Total_Amount)
VALUES (304,4,TO_DATE('2026-09-04','YYYY-MM-DD'),'DELIVERED',1499);

INSERT INTO Orders (Order_ID,Customer_ID,Order_Date,Order_Status,Total_Amount)
VALUES (305,5,TO_DATE('2026-09-05','YYYY-MM-DD'),'CANCELLED',2499);

6.Order_Item

INSERT INTO Order_Item (Order_Item_ID,Order_ID,Product_ID,Quantity,Unit_Price)
VALUES (401,301,201,1,2499);

INSERT INTO Order_Item (Order_Item_ID,Order_ID,Product_ID,Quantity,Unit_Price)
VALUES (402,302,202,1,3999);

INSERT INTO Order_Item (Order_Item_ID,Order_ID,Product_ID,Quantity,Unit_Price)
VALUES (403,303,203,1,799);

```
INSERT INTO Order_Item (Order_Item_ID,Order_ID,Product_ID,Quantity,Unit_Price)
VALUES (404,303,204,1,650);

INSERT INTO Order_Item (Order_Item_ID,Order_ID,Product_ID,Quantity,Unit_Price)
VALUES (405,304,205,1,1499);

INSERT INTO Order_Item (Order_Item_ID,Order_ID,Product_ID,Quantity,Unit_Price)

VALUES (406,305,206,1,2499);
```

7.Payment

```
INSERT INTO Payment
(Payment_ID,Order_ID,Payment_Date,Amount,Payment_Method,Payment_Status)
VALUES (501,301,TO_DATE('2026-09-01','YYYY-MM-DD'),2499,'UPI','PENDING');

INSERT INTO Payment
(Payment_ID,Order_ID,Payment_Date,Amount,Payment_Method,Payment_Status)
VALUES (502,302,TO_DATE('2026-09-02','YYYY-MM-DD'),3999,'CARD','PAID');

INSERT INTO Payment
(Payment_ID,Order_ID,Payment_Date,Amount,Payment_Method,Payment_Status)
VALUES (503,303,TO_DATE('2026-09-03','YYYY-MM-DD'),1449,'UPI','PAID');

INSERT INTO Payment
(Payment_ID,Order_ID,Payment_Date,Amount,Payment_Method,Payment_Status)
VALUES (504,304,TO_DATE('2026-09-04','YYYY-MM-DD'),1499,'CASH','PAID');

INSERT INTO Payment
(Payment_ID,Order_ID,Payment_Date,Amount,Payment_Method,Payment_Status)
VALUES (505,305,TO_DATE('2026-09-05','YYYY-MM-DD'),2499,'CARD','REFUNDED');
```

8. Shipment

```
INSERT INTO Shipment (Shipment_ID,Order_ID,Shipping_Address,Shipment_Date,Delivery_Status)
VALUES (601,301,'12 Main Road, Kakinada, Andhra Pradesh','2026-09-02','Processing');

INSERT INTO Shipment (Shipment_ID,Order_ID,Shipping_Address,Shipment_Date,Delivery_Status)
VALUES (602,302,'25 Beach Road, Visakhapatnam, Andhra Pradesh','2026-09-03','Shipped');

INSERT INTO Shipment (Shipment_ID,Order_ID,Shipping_Address,Shipment_Date,Delivery_Status)
VALUES (603,303,'45 MG Road, Vijayawada, Andhra Pradesh','2026-09-04','Shipped');

INSERT INTO Shipment (Shipment_ID,Order_ID,Shipping_Address,Shipment_Date,Delivery_Status)
VALUES (604,304,'10 Station Road, Rajahmundry, Andhra Pradesh','2026-09-05','Delivered');

INSERT INTO Shipment (Shipment_ID,Order_ID,Shipping_Address,Shipment_Date,Delivery_Status)
VALUES (605,305,'18 Banjara Road, Hyderabad, Telangana','2026-09-06','Cancelled');
```

10. SQL QUERIES & REPORTS

Q1. Display all customers

SELECT * FROM Customer;

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q01 - Display all customers

SQL> SELECT * FROM Customer;

OUTPUT:
 Customer_ID  |  Customer_Name  |  Email            |  Phone_No    |  Address
--------------+-----------------+-------------------+--------------+--------------------------------
 1            |  Rahul Kumar    |  rahul@gmail.com  |  9876543210  |  Kakinada, Andhra Pradesh
 2            |  Priya Sharma   |  priya@gmail.com  |  9876543211  |  Visakhapatnam, Andhra Pradesh
5 row(s) selected.
```

Q2: Display all addresses

SELECT * FROM Address;

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q02 - Display all addresses

SQL> SELECT * FROM Address;

OUTPUT:
 address_id  |  customer_id  |  address_line     |  city           |  state           |  postal_code  |  country
-------------+---------------+-------------------+-----------------+------------------+---------------+----------
 101         |  1            |  12 Main Road     |  Kakinada       |  Andhra Pradesh  |  533001       |  India
 102         |  2            |  25 Beach Road    |  Visakhapatnam  |  Andhra Pradesh  |  530001       |  India
5 row(s) selected.
```

Q3. Customers with email

SELECT Customer_ID, Customer_Name, Email FROM Customer WHERE Email IS NOT NULL;

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q03 - Customers with email

SQL> SELECT Customer_ID, Customer_Name, Email FROM Customer WHERE Email IS NOT NULL;

OUTPUT:
 Customer_ID  |  Customer_Name  |  Email
--------------+-----------------+------------------
1             |  Rahul Kumar    |  rahul@gmail.com
2             |  Priya Sharma   |  priya@gmail.com
5 row(s) selected.
```

Q4. Display addresses for a specific customer

SELECT * FROM Address WHERE Customer_ID=1;

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q04 - Display addresses for a specific customer

SQL> SELECT * FROM Address WHERE Customer_ID=1;

OUTPUT:
 address_id  |  customer_id  |  address_line  |  city      |  state           |  postal_code  |  country
-------------+---------------+----------------+------------+------------------+---------------+----------
 101         |  1            |  12 Main Road  |  Kakinada  |  Andhra Pradesh  |  533001       |  India
1 row(s) selected.
```

Q5. Customers with their addresses

SELECT c.Customer_ID, c.Customer_Name, a.Address_Line, a.City, a.State, a.Postal_Code FROM Customer c JOIN Address a ON c.Customer_ID=a.Customer_ID;

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q05 - Customers with their addresses

SQL> SELECT c.Customer_ID, c.Customer_Name, a.Address_Line, a.City, a.State, a.Postal_Code FRO
     JOIN Address a ON c.Customer_ID=a.Customer_ID;

OUTPUT:
 Customer_ID  |  Customer_Name  |  Address_Line     |  City           |  State           |  Po
--------------+-----------------+-------------------+-----------------+------------------+----
 101          |  Rahul Kumar    |  12 Main Road     |  Kakinada       |  Andhra Pradesh  |  53
 102          |  Priya Sharma   |  25 Beach Road    |  Visakhapatnam  |  Andhra Pradesh  |  53
5 row(s) selected.
```

Q6. Display addresses by city

SELECT * FROM Address WHERE City='Kakinada';

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q06 - Display addresses by city

SQL> SELECT * FROM Address WHERE City='Kakinada';

OUTPUT:
 address_id  |  customer_id  |  address_line  |  city      |  state           |  postal_code  |  country
-------------+---------------+----------------+------------+------------------+---------------+----------
 101         |  1            |  12 Main Road  |  Kakinada  |  Andhra Pradesh  |  533001       |  India
1 row(s) selected.
```

Q7. Display addresses by state

SELECT * FROM Address WHERE State='Andhra Pradesh';

Output :

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q07 - Display addresses by state

SQL> SELECT * FROM Address WHERE State='Andhra Pradesh';

OUTPUT:
 address_id  |  customer_id  |  address_line     |  city           |  state           |  postal_
-------------+---------------+-------------------+-----------------+------------------+---------
 101         |  1            |  12 Main Road     |  Kakinada       |  Andhra Pradesh  |  533001
 102         |  2            |  25 Beach Road    |  Visakhapatnam  |  Andhra Pradesh  |  530001
4 row(s) selected.
```

Q8. Count total addresses

SELECT COUNT(*) AS Total_Addresses FROM Address

;

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q08 - Count total addresses

SQL> SELECT COUNT(*) AS Total_Addresses FROM Address;

OUTPUT:
 TOTAL_ADDRESSES
-----------------
1
5 row(s) selected.
```

Q9. Number of addresses per customer

SELECT Customer_ID, COUNT(*) AS Address_Count FROM Address GROUP BY Customer_ID;

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q09 - Number of addresses per customer

SQL> SELECT Customer_ID, COUNT(*) AS Address_Count FROM Address GROUP BY Customer_ID;

OUTPUT:
 Customer_ID  |  Address_Count
--------------+----------------
5 row(s) selected.
```

Product & Category

Q10. Display all product

SELECT * FROM Product;

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q10 - Display all product

SQL> SELECT * FROM Product;

OUTPUT:
 Product_ID  |  Product_Name           |  Price  |  Stock_Qty  |  Category_ID
-------------+-------------------------+---------+-------------+--------------
 201         |  Wireless Headphones    |  2499   |  50         |  1
 202         |  Smart Watch            |  3999   |  30         |  1
6 row(s) selected.
```

Q11. Products above a price

SELECT * FROM Product WHERE Price > 1000;

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q11 - Products above a price

SQL> SELECT * FROM Product WHERE Price > 1000;

OUTPUT:
 Product_ID  |  Product_Name         |  Price  |  Stock_Qty  |  Category_ID
-------------+-----------------------+---------+-------------+--------------
 201         |  Wireless Headphones  |  2499   |  50         |  1
 202         |  Smart Watch          |  3999   |  30         |  1
4 row(s) selected.
```

Q12. Products sorted by price

SELECT * FROM Product ORDER BY Price DESC;
Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q12 - Products sorted by price

SQL> SELECT * FROM Product ORDER BY Price DESC;

OUTPUT:
 Product_ID  |  Product_Name           |  Price  |  Stock_Qty  |  Category_ID
-------------+-------------------------+---------+-------------+--------------
 202         |  Smart Watch            |  3999   |  30         |  1
 201         |  Wireless Headphones    |  2499   |  50         |  1
6 row(s) selected.
```

Q13. Products by category

SELECT p.Product_Name, c.Category_Name FROM Product p JOIN Category c ON p.Category_ID=c.Category_ID;
Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q13 - Products by category

SQL> SELECT p.Product_Name, c.Category_Name FROM Product p JOIN Category c ON p.Category_ID=c.Category_ID;

OUTPUT:
 Product_Name           |  Category_Name
------------------------+-----------------
 Wireless Headphones    |  Electronics
 Smart Watch            |  Electronics
6 rows selected.
```

Q14. Number of products by category

SELECT Category_ID, COUNT(*) FROM Product GROUP BY Category_ID;
Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q14 - Number of products by category

SQL> SELECT Category_ID, COUNT(*) FROM Product GROUP BY Category_ID;

OUTPUT:
 Category_ID  |  COUNT(*)
--------------+-----------
5 1          |  2
2 row(s) selected  1
```

Q15. Products with low stock

SELECT * FROM Product WHERE Stock_Qty < 10;
Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q15 - Products with low stock

SQL> SELECT * FROM Product WHERE Stock_Qty < 10;

OUTPUT:
 Product_ID  |  Product_Name     |  Price  |  Stock_Qty  |  Category_ID
-------------+-------------------+---------+-------------+--------------
205          |  Electric Kettle  |  1499   |  8          |  4
1 row(s) selected.
```

Q16. Search product by keyword

SELECT * FROM Product WHERE UPPER(Product_Name) LIKE '%PHONE%';
Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q16 - Search product by keyword

SQL> SELECT * FROM Product WHERE UPPER(Product_Name) LIKE '%PHONE%';

OUTPUT:
 Product_ID  |  Product_Name         |  Price  |  Stock_Qty  |  Category_ID
-------------+-----------------------+---------+-------------+--------------
201          |  Wireless Headphones  |  2499   |  50         |  1
1 row(s) selected.
```

Q17. Average product price

SELECT AVG(Price) AS Average_Price FROM Product;
Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q17 - Average product price

SQL> SELECT AVG(Price) AS Average_Price FROM Product;

OUTPUT:
 AVERAGE_PRICE
---------------
1990.83
1 row(s) selected.
```

Q18. Maximum product price

SELECT MAX(Price) AS Max_Price FROM Product;
Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q18 - Maximum product price

SQL> SELECT MAX(Price) AS Max_Price FROM Product;

OUTPUT:
 MAX_PRICE
-----------
3999
1 row(s) selected.
```

Q19. Minimum product price

SELECT MIN(Price) AS Min_Price FROM Product;
Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q19 - Minimum product price

SQL> SELECT MIN(Price) AS Min_Price FROM Product;

OUTPUT:
 MIN_PRICE
-----------
650
1 row(s) selected.
```

Orders & Order Items

Q20. Orders by status

SELECT * FROM Orders WHERE Order_Status='PLACED';

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q20 - Orders by status

SQL> SELECT * FROM Orders WHERE Order_Status='PLACED';

OUTPUT:
 Order_ID  |  Customer_ID  |  Order_Date  |  Order_Status  |  Total_Amount
-----------+---------------+--------------+----------------+---------------
301        |  1            |  2026-09-01  |  PLACED        |  2499
1 row(s) selected.
```

Q21. Orders with customer names

SELECT o.Order_ID, c.Customer_Name, o.Total_Amount FROM Orders o JOIN Customer c ON o.Customer_ID=c.Customer_ID;

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q21 - Orders with customer names

SQL> SELECT o.Order_ID, c.Customer_Name, o.Total_Amount FROM Orders o JOIN Customer c ON
     o.Customer_ID=c.Customer_ID;

OUTPUT:
 Order_ID  |  Customer_Name  |  Total_Amount
-----------+-----------------+---------------
 301       |  Rahul Kumar    |  2499
 302       |  Priya Sharma   |  3999
5 row(s) selected.
```

Q22. Order items with products

SELECT oi.Order_ID, p.Product_Name, oi.Quantity FROM Order_Item oi JOIN Product p ON oi.Product_ID=p.Product_ID;

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q22 - Order items with products

SQL> SELECT oi.Order_ID, p.Product_Name, oi.Quantity FROM Order_Item oi JOIN Product p ON
     oi.Product_ID=p.Product_ID;

OUTPUT:
 Order_ID  |  Product_Name           |  Quantity
-----------+-------------------------+-----------
 301       |  Wireless Headphones    |  1
 302       |  Smart Watch            |  1
6 row(s) selected.
```

Q23. Count total orders

SELECT COUNT(*) AS Total_Orders FROM Orders;

output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q23 - Count total orders

SQL> SELECT COUNT(*) AS Total_Orders FROM Orders;

OUTPUT:
 TOTAL_ORDERS
--------------
1 5 row(s) selected.
```

Q24. Orders per customer

SELECT Customer_ID, COUNT(*) AS Order_Count FROM Orders GROUP BY Customer_ID;

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q24 - Orders per customer

SQL> SELECT Customer_ID, COUNT(*) AS Order_Count FROM Orders GROUP BY Customer_ID;

OUTPUT:
 Customer_ID  |  Order_Count
--------------+--------------
1             | 1
2             | 1
5 row(s) selected.
```

Q25. Units sold per product

SELECT Product_ID, SUM(Quantity) AS Units_Sold FROM Order_Item GROUP BY Product_ID;

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q25 - Units sold per product

SQL> SELECT Product_ID, SUM(Quantity) AS Units_Sold FROM Order_Item GROUP BY Product_ID;

OUTPUT:
 Product_ID  |  Units_Sold
-------------+-------------
201          | 1
202          | 1
6 row(s) selected.
```

Q26. Order status summary

SELECT Order_Status, COUNT(*) FROM Orders GROUP BY Order_Status;

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q26 - Order status summary

SQL> SELECT Order_Status, COUNT(*) FROM Orders GROUP BY Order_Status;

OUTPUT:
 Order_Status  |  COUNT(*)
---------------+-----------
CANCELLED      | 1
DELIVERED      | 1
5 rows selected.
```

Q27. Customer order history

SELECT c.Customer_Name, o.Order_ID, o.Order_Date, o.Order_Status FROM Customer c JOIN Orders o ON c.Customer_ID=o.Customer_ID;

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q27 - Customer order history

SQL> SELECT c.Customer_Name, o.Order_ID, o.Order_Date, o.Order_Status FROM Customer c JOIN Orders o ON
     c.Customer_ID=o.Customer_ID;

OUTPUT:
 Customer_Name  |  Order_ID  |  Order_Date  |  Order_Status
----------------+------------+--------------+---------------
 Rahul Kumar    |  301       |  2026-09-01  |  PLACED
 Priya Sharma   |  302       |  2026-09-02  |  PAID
5 row(s) selected.
```

Q28. Orders above amount

SELECT * FROM Orders WHERE Total_Amount > 2499;

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q28 - Orders above amount

SQL> SELECT * FROM Orders WHERE Total_Amount > 2499;

OUTPUT:
 Order_ID  |  Customer_ID  |  Order_Date  |  Order_Status  |  Total_Amount
-----------+---------------+--------------+----------------+---------------
 302       |  2            |  2026-09-02  |  PAID          |  3999
1 row(s) selected.
```

Q29. Average order value

SELECT AVG(Total_Amount) AS Average_Order_Value FROM Orders;

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q29 - Average order value

SQL> SELECT AVG(Total_Amount) AS Average_Order_Value FROM Orders;

OUTPUT:
 AVERAGE_ORDER_VALUE
---------------------
 2389.00
1 row(s) selected.
```

Q30. Highest-value order

SELECT * FROM Orders WHERE Total_Amount=(SELECT MAX(Total_Amount) FROM Orders);

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q30 - Highest-value order

SQL> SELECT * FROM Orders WHERE Total_Amount=(SELECT MAX(Total_Amount) FROM Orders);

OUTPUT:
 Order_ID  |  Customer_ID  |  Order_Date  |  Order_Status  |  Total_Amount
-----------+---------------+--------------+----------------+---------------
302         |  2            |  2026-09-02  |  PAID          |  3999
1 row(s) selected.
```

Q31. Customers with orders

SELECT DISTINCT c.Customer_Name FROM Customer c JOIN Orders o ON c.Customer_ID=o.Customer_ID;

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q31 - Customers with orders

SQL> SELECT DISTINCT c.Customer_Name FROM Customer c JOIN Orders o ON c.

OUTPUT:
 Customer_Name
---------------
Rahul Kumar
Priya Sharma
5 row(s) selected.
```

Payment & Shipment

Q32. Payment details

SELECT o.Order_ID, p.Amount, p.Payment_Method, p.Payment_Status FROM Orders o JOIN Payment p ON o.Order_ID=p.Order_ID;

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q32 - Payment details

SQL> SELECT o.Order_ID, p.Amount, p.Payment_Method, p.Payment_Status FROM Orders o JOIN Payment p ON
     o.Order_ID=p.Order_ID;

OUTPUT:
 Order_ID  |  Amount  |  Payment_Method  |  Payment_Status
-----------+----------+------------------+-----------------
 301       |  2499    |  UPI             |  PENDING
 302       |  3999    |  CARD            |  PAID
5 row(s) selected
```

Q33. Pending payments

SELECT * FROM Payment WHERE Payment_Status='Paid';

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q33 - Pending payments

SQL> SELECT * FROM Payment WHERE Payment_Status='Paid';

OUTPUT:
 Payment_ID  |  Order_ID  |  Payment_Date  |  Amount  |  Payment_Method  |  Payment_Status
-------------+------------+----------------+----------+------------------+-----------------
 502         |  302       |  2026-09-02    |  3999    |  CARD            |  PAID
 503         |  303       |  2026-09-03    |  1449    |  UPI             |  PAID
3 row(s) selected
```

Q34. Total delivered sales

SELECT SUM(Total_Amount) AS Total_Sales FROM Orders WHERE Order_Status='DELIVERED';

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q34 - Total delivered sales

SQL> SELECT SUM(Total_Amount) AS Total_Sales FROM Orders WHERE Order_Status='DELIVERED';

OUTPUT:
 TOTAL_SALES
-------------
        1499
1 row(s) selected.
```

Q35. Shipment details

SELECT o.Order_ID, s.Shipping_Address, s.Delivery_Status FROM Orders o JOIN Shipment s ON o.Order_ID=s.Order_ID;

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q35 - Shipment details

SQL> SELECT o.Order_ID, s.Shipping_Address, s.Delivery_Status FROM Orders o JOIN Shipment s ON
     o.Order_ID=s.Order_ID;

OUTPUT:
 Order_ID  |  Shipping_Address                              |  Delivery_Status
-----------+------------------------------------------------+------------------
       301 |  12 Main Road, Kakinada, Andhra Pradesh        |  Processing
       302 |  25 Beach Road, Visakhapatnam, Andhra Pradesh  |  Shipped
5 row(s) selected.
```

Q36. Delivered shipments

SELECT * FROM Shipment WHERE Delivery_Status='Delivered';

Output:

```
SQL*PLUS  |  ONLINE SHOPPING SYSTEM

ONLINE SHOPPING SYSTEM
10. SQL QUERIES & REPORTS
Q36 - Delivered shipments

SQL> SELECT * FROM Shipment WHERE Delivery_Status='Delivered';

OUTPUT:
 Shipment_ID  |  Order_ID  |  Shipping_Address                              |  Shipment_Date  |  Delivery_Stat
--------------+------------+------------------------------------------------+-----------------+---------------
          604 |        304 |  10 Station Road, Rajahmundry, Andhra Pradesh  |  2026-09-05     |  Delivered
1 row(s) selected.
```

11. CONCLUSION

The Online Shopping System provides a structured relational database solution for managing customers, categories, products, orders, order items, payments and shipments. The design organizes business information into related relations and applies primary keys, foreign keys, uniqueness rules, NOT NULL conditions and CHECK constraints to support data integrity. The normalization process up to 3NF reduces redundancy and update anomalies. The SQL implementation and query set demonstrate insertion, retrieval, joins, aggregation and operational reporting.

12. FUTURE ENHANCEMENTS

- Role-based access for administrators, customers and ope
- rational staff.
- Inventory reservation and automatic stock updates.
- Coupon, discount, tax and promotion management.
- Returns, cancellations and refund processing.
- Automated email/SMS/order notifications.
- Analytics dashboards for sales and customer behavior.
- Audit logging, backup, recovery and stronger database security.
- Integration with web and mobile online shopping applications.

13. REFERENCES

1. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, McGraw Hill.
2. Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, Pearson.
3. Supplied DBMS Cornerstone Project documentation used as the structural reference for this documentation.